Guitare électrique : Etude de l'influence du type de bobinage dans le micro

Christophe Morel

Lycée Ferdinand Buisson, dit ''la Nat'', CPGE PT

Annexes

Code Python

Table des matières

TABLE DES MATIERES 2

I. MODELISATION.PY 3

I.I. FONCTION UTILES DANS L'ENTIERETE DU TIPE 3

I.I.I. save_plot() 3

I.I.II. Normalize_signal() 4

I.I.II. compute_and_normalize_fft() 4

I.I.II. extraire _pics_separes() 4

I.II. FONCTION UTILES POUR LA MODELISATION ELECTRIQUE 5

I.II.I. Z_micro() 5

I.II.II. Matrice_Jacobienne() 5

I.II.III. residus() 5

I.II.IV. Levenberg_marquardt() 6

I.II.V. calcul_bode() 6

I.III. FONCTIONS UTILES POUR LA MODELISATION ACOUSTIQUE 7

I.III.I. fem_singlecoil() 7

I.III.II. fem_humbucker() 7

I.IV. FONCTION UTILE POUR LA MODELISATION MAGNETIQUE 8

I.IV.I. fem_par_s() 8

I.IV.II. fem_par_h() 8

II. REPONSE_ELEC.PY 9

III. REPONSE_MECANIQUE.PY 11

IV. REPONSE_MAGNETIQUE.PY 14

I. Modelisation.py

I.I. Fonction utiles dans l'entièreté du TIPE

I.I.I. save_plot()

```python
def save_plot(fig_path: str, *plot_calls, title:str="", xlabel:str="", ylabel:str="", ylim=None, xlim=None, legend:bool=True, log:bool=False, vspan:bool=False):
    """
    Enregistre une figure matplotlib avec des options de personnalisation.

    :param fig_path: Chemin vers le fichier de sauvegarde
    :type fig_path: str
    :param plot_calls: Appels de tracé sous la forme (x, y, kwargs)
    :type plot_calls: tuple
    :param title: Titre de la figure
    :type title: str
    :param xlabel: nom de l'axe des x
    :type xlabel: str
    :param ylabel: nom de l'axe des y
    :type ylabel: str
    :param ylim: Limites de l'axe des y
    :type ylim: tuple
    :param legend: presence de la legende
    :type legend: bool
    :param log: presence d'une echelle logarithmique sur l'axe des x
    :type log: bool
    :param vspan: presence des zones de frequences fondamentales et harmoniques
    :type vspan: bool
    """
    plt.figure(figsize=(6, 5))
    handles_curves = []
    z_handles = []
    if vspan:
        z2 = plt.axvspan(82.41, 1174.66, color="#F0E68C", alpha=0.3,
                         label="Frequences fondamentales", zorder=0)
        z1 = plt.axvspan(1174.66, 5*1174.66, facecolor="none", alpha=0.5, hatch="////",
                         edgecolor="#424242", label="Frequences harmoniques", zorder=0)
        z_handles = [z1, z2]

    for call in plot_calls:
        x, y, kwargs = call
        # On recupère l'objet ligne retourne par plot (c'est un tuple, d'où le [0])
        line, = plt.plot(x, y, **kwargs)
        handles_curves.append(line)

    if title: plt.title(title)
    if xlabel: plt.xlabel(xlabel)
    if ylabel: plt.ylabel(ylabel)
    if ylim is not None: plt.ylim(ylim)
    if xlim is not None: plt.xlim(xlim)

    plt.grid(True, alpha=0.6) # Grid plus discrète pour le contraste
    if log: plt.xscale('log')

    if legend:
        # 1. Legende des courbes (Interne)
        leg_courbes = plt.legend(handles=handles_curves, loc='best', framealpha=1)

        # 2. Legende des zones (Externe à droite)
        if vspan:
            plt.gca().add_artist(leg_courbes)
            plt.legend(handles=z_handles, title="Zones d'etude", bbox_to_anchor=(0.5, -0.18), loc='upper center', ncol=2, borderaxespad=0.)

    # Ajustement automatique pour ne pas couper la legende externe
    plt.tight_layout()
    plt.savefig(fig_path, dpi=300) # dpi=200 pour une nettete accrue en projection
    plt.close()

```

I.I.II. Normalize_signal()

```python
def normalize_signal(signal: np.ndarray):
    """
    Normalise un signal entre -1 et 1.

    :param signal: Signal à normaliser
    :type signal: np.ndarray
    """
    max_val = np.max(np.abs(signal))
    if max_val == 0:
        return signal
    return signal / max_val
```

I.I.II. compute_and_normalize_fft()

```python
def compute_and_normalize_fft(signal: np.ndarray, Ne_fft: int):
    """
    Calcule et normalise l'amplitude du spectre de Fourier d'un signal.

    :param signal: Signal à analyser
    :type signal: np.ndarray
    :param Ne_fft: Nombre de points dans la FFT
    :type Ne_fft: int
    """

    fft = np.fft.fft(signal)
    amp = np.abs(fft) / Ne_fft
    return amp
```

I.I.II. extraire_pics_separes()

```python
def extraire_pics_separes(frequences, amplitudes, prominence=0.05, distance=10):
    """
    Extrait les pics significatifs d'un spectre de Fourier en utilisant la proeminence.
    Args:
        frequences (np.ndarray): Tableau des frequences
        amplitudes (np.ndarray): Tableau des amplitudes correspondantes
        prominence (float): Hauteur relative du pic par rapport à sa base locale.
                            Permet de detecter les petits pics aigus sans prendre le bruit fort.
        distance (int): Distance minimale entre deux pics pour les considérer comme séparés.

    """
    # Utilisation de 'prominence' au lieu de 'height' pour le problème d'amplitude variable
    indices, _ = find_peaks(amplitudes, prominence=prominence, distance=distance)

    # Extraction et conversion en listes
    liste_frequences = frequences[indices].tolist()
    liste_amplitudes = amplitudes[indices].tolist()

    return np.array(liste_frequences), np.round(liste_amplitudes, 3)

```

I.II. Fonction utiles pour la modélisation électrique

I.II.I. Z_micro()

```python
def Z_micro(X:np.ndarray, rs:float, w:np.ndarray)-> np.ndarray:
    """Calcul l'impedance d'un micro de guitare en une pulsation w

    Args:
        X (np.ndarray): Vecteur des caracteristique electrique interne du micro [L, C, Rp] avec L l'inductance [H], C la capacite parallèle [nF]et
            Rp la resistance parallèle [kOhm] modelisant les courants de foucauts
        rs (float): Resistance serie (DC) du micro. Peut etre mesure avec un Ohmmetre
        w (np.ndarray): Tableau des pulsation w

    Returns:
        np.darray: Tableau Numpy contenant le module de l'impedance complexe |Z| en chaque pulsation w
    """
    L, C_n, Rp_k = X
    C, Rp = C_n * 1e-9, Rp_k * 1e3

    # Parties reelle et imaginaire de l'admittance Y
    den_L = rs**2 + (L * w)**2
    Y_re = (1 / Rp) + (rs / den_L)
    Y_im = (C * w) - (L * w / den_L)

    return 1 / np.sqrt(Y_re**2 + Y_im**2)
```

I.II.II. Matrice_Jacobienne()

```python
def Matrice_Jacobienne(X:np.ndarray, rs:float, f:np.ndarray)->np.ndarray:
    """Basee sur l'algorythme de Levenberg_Marquardt, cette fonction retourne la matrice Jacobienne J

    Args:
        X (np.ndarray): Vecteur des caracteristique electrique interne du micro [L, C, Rp] avec L l'inductance [H], C la capacite parallèle [nF]et
            Rp la resistance parallèle [kOhm] modelisant les courants de foucauts
        rs (float):Resistance serie (DC) du micro. Peut etre mesure avec un Ohmmetre
        f (np.ndarray): Tableau des frequence f

    Returns:
        np.ndarray: Matrice Jacobienne definie tel que chaque colonne i contient les derivees partielles par rapport a la variable situe en i dans X pour toute
            pulsation w
    """
    w = 2 * np.pi * f
    L, C_n, Rp_k = X
    C, Rp = C_n * 1e-9, Rp_k * 1e3

    # Variables intermediaires
    den_L = rs**2 + (L * w)**2
    Y_re = (1 / Rp) + (rs / den_L)
    Y_im = (C * w) - (L * w / den_L)
    Z = 1 / np.sqrt(Y_re**2 + Y_im**2)
    Z3 = Z**3

    # Derivees partielles de Y
    dYre_dL = - (rs * 2 * L * w**2) / (den_L**2)
    dYim_dL = - (w * (rs**2 - (L * w)**2)) / (den_L**2)
    dZ_dL = - Z3 * (Y_re * dYre_dL + Y_im * dYim_dL)

    dYim_dCn = w * 1e-9
    dZ_dCn = - Z3 * (Y_im * dYim_dCn)

    dYre_dRpn = - 1 / (1000 * Rp_k**2)
    dZ_dRpk = - Z3 * (Y_re * dYre_dRpn)

    return np.column_stack((dZ_dL, dZ_dCn, dZ_dRpk))

```

I.II.III. residus()

```python
def residus(X:np.ndarray, rs:float, f:np.ndarray, Z_exp:np.ndarray)-> np.ndarray:
    """Calcul l'ecart (l'erreur) entre l'impedance calcule theoriquement et l'impedance reelle experimentale

    Args:
        X (np.ndarray): Vecteur des caracteristique electrique interne du micro [L, C, Rp] avec L l'inductance [H], C la capacite parallèle [nF]et
            Rp la resistance parallèle [kOhm] modelisant les courants de foucauts
        rs (float):Resistance serie (DC) du micro. Peut etre mesure avec un Ohmmetre
        f (np.ndarray): Tableau des frequence f
        Z_exp (np.ndarray): Tableau Numpy des impedances experimentale releve en Ohm

    Returns:
        np.ndarray: Tableau Numpy des distances separant chaque point theorique au point experimentaux
    """
    return Z_exp - Z_micro(X, rs, 2 * np.pi * f)
```

I.II.IV. Levenberg_marquardt()

```python
def Levenberg_Marquardt(X_init:np.ndarray, rs:float, f:np.ndarray, Z_exp:np.ndarray, N:int, learning_rate:float):
    """Algorythme de Levenberg-Marquardt pour l'optimisation non-lineaire des parametres electriques du micro

    Args:
        X_init (np.ndarray): Vecteur initial des caracteristique electrique interne du micro [L, C, Rp] avec L l'inductance [H], C la capacite parallèle [nF]et
            Rp la resistance parallèle [kOhm] modelisant les courants de foucauts
        rs (float): Resistance serie (DC) du micro. Peut etre mesure avec un Ohmmetre
        f (np.ndarray): Tableau des frequence f
        Z_exp (np.ndarray): Tableau Numpy des impedances experimentale releve en Ohm
        N (int): Nombre d'iterations de l'algorithme

    Returns:
        np.ndarray: Vecteur optimisé des caracteristique electrique interne du micro [L, C, Rp] avec L l'inductance [H], C la capacite parallèle [nF]et
            Rp la resistance parallèle [kOhm] modelisant les courants de foucauts
    """
    X = np.array(X_init, dtype=float)
    alpha = learning_rate
    v = 3
    r = None
    eps = None

    for _ in range(N):
        r = residus(X, rs, f, Z_exp)
        eps = np.sum(r**2)
        J = Matrice_Jacobienne(X, rs, f)

        H = J.T @ J
        g = J.T @ r

        # Amortissement de Marquardt
        H_lm = H + alpha * np.diag(np.diag(H) + 1e-8)

        try:
            delta = np.linalg.solve(H_lm, g)
        except np.linalg.LinAlgError:
            break

        X_new = X + delta
        # Contrainte physique : paramètres toujours positifs
        X_new = np.maximum(X_new, [1e-3, 1e-3, 1e-3])
        eps_new = np.sum(residus(X_new, rs, f, Z_exp)**2)
        if eps_new < eps:
            alpha /= v
            X = X_new
            if np.linalg.norm(delta) / np.linalg.norm(X) < 1e-12:
                break
        else:
            alpha *= v

    return X, eps_new
```

I.II.V. calcul_bode()

```python
def calcul_bode(X:np.ndarray, rs:float, f:np.ndarray, Ramp:float=1e6)-> tuple[np.ndarray, np.ndarray]:
    """Calcul la fonction de transfert en gain et phase d'un micro de guitare en fonction de ses parametres internes

    Args:
        X (np.ndarray): Vecteur optimisé des caracteristique electrique interne du micro [L, C, Rp] avec L l'inductance [H], C la capacite parallèle [nF]et
            Rp la resistance parallèle [kOhm] modelisant les courants de foucauts
        rs (float): Resistance serie (DC) du micro. Peut etre mesure avec un Ohmmetre
        f (np.ndarray): Tableau des frequence f
        Ramp (float, optional): Resistance d'entree de l'amplificateur. Defaults to 1e6.

    Returns:
        tuple[np.ndarray, np.ndarray]: Deux tableaux Numpy contenant respectivement le gain (en dB) et la phase (en degres) de la fonction de transfert du micro
    """
    w = 2 * np.pi * f
    L, C, Rp = X[0], X[1] * 1e-9, X[2] * 1e3

    # Impedance serie (la bobine qui genère le signal)
    Z_ser = rs + 1j * L * w

    # Impedance parallèle (ce qui "shunte" le signal vers la masse)
    Y_para = (1 / Rp) + (1 / Ramp) + (1j * C * w)
    Z_para = 1 / Y_para

    # Fonction de transfert correcte
    H = Z_para / (Z_ser + Z_para)

    gain = 20 * np.log10(np.abs(H))
    phase = np.angle(H, deg=True)
    return gain, phase
```

I.III. Fonctions utiles pour la modélisation acoustique

I.III.I. fem_singlecoil()

```python
def fem_singlecoil(n: int, N: int, xm: float, t: np.ndarray, A0: float, L: float, T: float, mu: float, x0: float, S: float, dBdy: float) -> np.ndarray:
    """Force electromotrice induite dans le micro singlecoil

    Args:
        n (int): Nombre de modes consideres
        N (int): Nombre de spires de la bobine du micro singlecoil
        xm (float): Position du micro sur la corde
        t (np.ndarray): Tableau des instants
        A0 (float): Amplitude initiale de la vibration
        L (float): Longueur de la corde
        T (float): Tension de la corde
        mu (float): Masse linéique de la corde
        x0 (float): Position de la perturbation
        S (float): Surface effective du micro
        dBdy (float): Variation du flux magnétique par unité de déplacement vertical

    Returns:
        np.ndarray: Tableau du signal de la force electromotrice induite dans le micro singlecoil
    """
    s = 0
    for i in range(1, n + 1):
        fn = (i / (2 * L)) * np.sqrt(T / mu)
        An = 2 * np.pi * fn * np.sin(i * np.pi * xm / L)
        Bn = (A0 * L**2.5 * np.sqrt(2)) / (i**2 * np.pi**2 * x0 * (L - x0)) * np.sin(i * np.pi * x0 / L)
        s += An * Bn * np.sin(2 * np.pi * fn * t)
    return N * S * dBdy * s
```

I.III.II. fem_humbucker()

```python
def fem_humbucker(n: int, N: int, x1: float, x2: float, t: np.ndarray, A0: float, L: float, T: float, mu: float, x0: float, S: float, dBdy: float) -> np.ndarray:
    """Force electromotrice induite dans le micro Humbucker

    Args:
        n (int): Nombre de modes consideres
        N (int): Nombre de spires de la bobine du micro Humbucker
        x1 (float): Position du 1er bobinage du micro sur la corde
        x2 (float): Position du 2ème bobinage du micro sur la corde
        t (np.ndarray): Tableau des instants
        A0 (float): Amplitude initiale de la vibration
        L (float): Longueur de la corde
        T (float): Tension de la corde
        mu (float): Masse linéique de la corde
        x0 (float): Position de la perturbation
        S (float): Surface effective du micro
        dBdy (float): Variation du flux magnétique par unité de déplacement vertical

    Returns:
        np.ndarray: Tableau du signal de la force electromotrice induite dans le micro Humbucker
    """
    s = 0
    for i in range(1, n + 1):
        fn = (i / (2 * L)) * np.sqrt(T / mu)
        An = 2 * np.pi * fn * (np.sin(i * np.pi * x1 / L) + np.sin(i * np.pi * x2 / L))
        Bn = (A0 * L**2.5 * np.sqrt(2)) / (i**2 * np.pi**2 * x0 * (L - x0)) * np.sin(i * np.pi * x0 / L)
        s += An * Bn * np.sin(2 * np.pi * fn * t)
    return N * S * dBdy * s
```

I.IV. Fonction utile pour la modelisation magnetique

I.IV.I. fem_par_s()

```python
def fem_par_s(N: int, fB: float, t: np.ndarray, B0: float, S: float, phi: float) -> np.ndarray:
    """Force electromotrice induite dans une bobine par une variation sinusoïdale d'un champ magnétique exterieu
r

    Args:
        N (int): Nombre de spires du micro
        fB (float): Fréquence de variation du champ magnétique
        t (np.ndarray): Tableau des instants
        B0 (float): Amplitude du champ magnétique
        S (float): Surface effective du micro
        phi (float): Phase initiale de la variation du champ magnétique

    Returns:
        np.ndarray: Tableau du signal de la force electromotrice induite dans une bobine

    Returns:
        np.ndarray: _description_
    """
    return N*S*2*np.pi*fB*B0*np.sin(2*np.pi*fB*t + phi)
```

I.IV.II. fem_par_h()

```python
def fem_par_h(N: int, x1: float, x2: float, fB: float, t: np.ndarray, B0: float, alpha: float, S: float) -> np.n
darray:
    """Force electromotrice induite dans un humbucker par une variation sinusoïdale d'un champ magnétique exteri
eur

    Args:
        N (int): Nombre de spires du micro
        x1 (float): Position du 1er bobinage du micro humbucker sur la corde
        x2 (float): Position du 2ème bobinage du micro humbucker sur la corde
        fB (float): Fréquence de variation du champ magnétique
        t (np.ndarray): Tableau des instants
        B0 (float): Amplitude du champ magnétique
        alpha (float): Gradient du champ magnétique
        S (float): Surface effective du micro

    Returns:
        np.ndarray: Tableau du signal de la force electromotrice induite dans un humbucker
    """
    return N*S*2*np.pi*fB*B0*alpha*abs(x1-x2)*np.sin(2*np.pi*fB*t)
```

II. Reponse_elec.py

```python
                                        #!####################!#
                                        #!   Bibliothèques    !#
                                        #!####################!#

print("\nImportation des bibliotheques...")
print("\nImportation des bibliotheques...")
import sys
import os
sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__), '../..')))
from library import Modelisation as mod
import numpy as np
import matplotlib.pyplot as plt
import logging
plt.close("all")
logging.basicConfig(level=logging.INFO, format='%(levelname)s: %(message)s')
print("Bibliotheques importees.")

                                        #!####################!#
                                        #!     Variables      !#
                                        #!####################!#

print("\nInitialisation des variables...")
Rs = 5.3e3          # Resistance du single coil en ohms
Rh = 4.52e3        # Resistance du humbucker en ohms
Ramp = 1e6         # Resistance de l'ampli en ohms
X0 = np.array([2, 0.5, 100])
LR = 0.01
Color_c = "#505050"
Color_h_test = "#008B8B"
Color_s_test = "#FF8C00"
ls_exp=':'
lw_exp=2
m_exp="o"
ms_exp=5
Color_h = "#000080"
Color_s = "#8B0000"
ls_th='-'
lw_th=2
```

```python
def lire_fichier(PATH):
    f = []
    Vin = []
    Vs = []
    Vh = []
    file_s2 = open(PATH)
    L_lignes = file_s2.readlines()

    for ligne in L_lignes[1:]:
        ligne = ligne.strip().split(',')
        ligne = [x.replace('\n', '') for x in ligne]
        ligne = [x.replace(',', '.') for x in ligne]
        Vin.append(float(ligne[0]))
        f.append(float(ligne[1]))
        Vs.append(float(ligne[2]))
        Vh.append(float(ligne[3]))

    return np.array(Vin), np.array(f), np.array(Vs), np.array(Vh)


print("Variables initialisees.")

                                        #!####################!#
                                        #! Lectures Fichiers  !#
                                        #!####################!#

Vin, f, Vs, Vh = lire_fichier("Python/data/raw/!Mesure_Impedance.csv")
f2, Vin2, Vs2, Vh2 = lire_fichier("Python/data/raw/!!Mesures_impedance_500Ohm_2_fev.csv")

Z_sc = Ramp * (Vs / (24000 - Vs))
Z_hb = Ramp * (Vh / (24000 - Vh))

Z_sc2 = 500 * (21.1/Vs2 - 1)
Z_hb2 = 500 * (21.1/Vh2 - 1)
```

```
                                        #!####################!#
                                        #!      Fonction      !#
                                        #!####################!#

Liste_S_sc = []
Liste_S_hb = []
liste_iter = []
Liste_Xsc = []
Liste_Xhb =[]
for i in range(1, 50):
    X_sc, S_sc = mod.Levenberg_Marquardt(X0, Rs, f, Z_sc, i, LR)
    X_hb, S_hb = mod.Levenberg_Marquardt(X0, Rh, f, Z_hb, i, LR)
    Liste_S_sc.append(S_sc)
    Liste_S_hb.append(S_hb)
    Liste_Xsc.append(X_sc)
    Liste_Xhb.append(X_hb)
    liste_iter.append(i)

Liste_S_sc = np.array(Liste_S_sc)
Liste_S_hb = np.array(Liste_S_hb)

X_sc, S_sc = mod.Levenberg_Marquardt(X0, Rs, f, Z_sc, 20, LR)
X_hb, S_hb = mod.Levenberg_Marquardt(X0, Rh, f, Z_hb, 20, LR)

print(f"SingleCoil: L={X_sc[0]:.2f}H, C={X_sc[1]:.2f}nF, Rp={X_sc[2]:.1f}kOhm")
print(f"Humbucker: L={X_hb[0]:.2f}H, C={X_hb[1]:.2f}nF, Rp={X_hb[2]:.1f}kOhm")

X_sc2, S_sc2 = mod.Levenberg_Marquardt(X0, Rs, f2, Z_sc2, 20, LR)
X_hb2, S_hb2 = mod.Levenberg_Marquardt(X0, Rh, f2, Z_hb2, 20, LR)

print(f"SingleCoil: L={X_sc2[0]:.2f}H, C={X_sc2[1]:.2f}nF, Rp={X_sc2[2]:.1f}kOhm")
print(f"Humbucker: L={X_hb2[0]:.2f}H, C={X_hb2[1]:.2f}nF, Rp={X_hb2[2]:.1f}kOhm")
```

```
mod.save_plot("Python/Exports/01-Reponse_electrique_frequetielle/Impedance.png",
                (f, Z_sc*1e-3, {'marker': m_exp, 'markersize': ms_exp, 'linestyle': ls_exp, 'linewidth': lw_exp, 'color': Color_s_test, 'label': "Micro SingleCoil"}),
                (f, Z_hb*1e-3, {'marker': m_exp, 'markersize': ms_exp, 'linestyle': ls_exp, 'linewidth': lw_exp, 'color': Color_h_test, 'label': "Micro Humbucker"}),
                (np.linspace(20, 100_000, 1000), mod.Z_micro(X_sc, Rs, 2*np.pi*np.linspace(20, 100_000, 1000))*1e-3, {'linestyle': ls_th, 'linewidth': lw_th, 'color': Color_s, 'label': "Micro SingleCoil - Prediction"}),
                (np.linspace(20, 100_000, 1000), mod.Z_micro(X_hb, Rh, 2*np.pi*np.linspace(20, 100_000, 1000))*1e-3, {'linestyle': ls_th, 'linewidth': lw_th, 'color': Color_h, 'label': "Micro Humbucker - Prediction"}),
                title=r"Impedances des micros $|Z|$ en fonction de la frequence",
                xlabel="Frequence [$Hz$]",
                ylabel=r"Impedance [$k\Omega$]",
                vspan=True,
                log=False,
                xlim=(0, 20_000),
                ylim=(0,300),
                legend=True)
```

III. Reponse_mecanique.py

```python
                                        #!####################!#
                                        #!   Bibliothèques    !#
                                        #!####################!#

print("\nImportation des bibliotheques...")
import os
import sys
sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__), '../..')))
from library import Modelisation as mod
import numpy as np
import matplotlib.pyplot as plt
import logging
plt.close("all")
logging.basicConfig(level=logging.INFO, format='%(levelname)s: %(message)s')
print("Bibliotheques importees.")

                                        #!####################!#
                                        #!    Constantes      !#
                                        #!####################!#

print("-" * 100)
print("\nInitialisation des constantes")
#* Constante pour code
tracer_fft = True
PATH_exp = "Python/Exports/02-Reponse_mecanique/"
#* Constante physique
rho = 1.68 * 10**(-8)   # Masse volumique du cuivre (Ohm.m)
mu0 = 4e-7 * np.pi      # Permeabilite magnetique du vide (H.m-1)
#* Parametre maquette
xm = 0.105              # Position du micro singlecoil
x1 = 0.140              # Position du micro humbucker 1
x2 = 0.160              # Position du micro humbucker 2
L = 0.845               # Longueur de la corde
mu = 9 * 10**(-3)       # Masse lineique de la corde
dBdy = 0.03             #? Gradient du champ magnetique (A verifier ???)
#* Parametre Micros
d = 0.063 * 10**(-3)    # Diametre du fil de cuivre
S = 7 * 10**(-4)        # Surface de la bobine du micro
#* Parametre Micro Singlecoil
RCC_s = 5.3 * 10**3     # Resistance DC micro singlecoil
lmoy_s = 0.147          #? Longueur moyenne du fil enroule
N_s = int(RCC_s * np.pi * d**2 / (4 * rho * lmoy_s))    # Nombre de spires du micro singlecoil
#* Parametre Micro Humbucker
RCC_h = 4.52 * 10**3    # Resistance DC micro humbucker
lmoy_h = lmoy_s         #? Longueur moyenne d'une bobine humbucker
N_h = 2*int(RCC_h * np.pi * d**2 / (8 * rho * lmoy_h))
#* Parametre variable
A = 70 * 10**(-4)          #? Amplitude initiale de vibration
x0 = 0.140                 #? Origine de la perturbation
F = 78.0                   #? Frequence de la note jouee
T = 4 * mu * (L * F)**2    #? Tension de la corde
#* Couleurs graphiques
Color_h_test = "#008B8B"
Color_s_test = "#FF8C00"
ls_exp='--'
m_exp="o"
ms_exp=5
Color_h = "#000080"
Color_s = "#8B0000"
ls_th='-'
lw_th=2
```

```
                                        #!####################!#
                                        #!      Get data      !#
                                        #!####################!#

path = "Python/data/raw/SingleCoil_corde_78,5.csv"
try:
    with open(path) as doc:
        L_lignes = doc.readlines()
    Test_s_fft = []
    Test_temps_s_fft = []
    for i in range(35200, 60200):
        values = L_lignes[i].strip().split(";")
        values = [v.replace(',', '.') for v in values]
        values = [float(v.replace("\n", "")) for v in values]
        Test_temps_s_fft.append(values[0])
        Test_s_fft.append(values[1])
    Test_temps_s_fft = np.array(Test_temps_s_fft)
    Test_temps_s_fft -= Test_temps_s_fft[0]
    Test_s_fft = np.array(Test_s_fft)
    print("Donnees de test Single Coil corde seule lues.")
except Exception as e:
    logging.error(f"Erreur lors de la lecture des donnees de test Single Coil corde seule : {e}")
```

```
print("\nCalcul des tensions induites...")
print("\nEchantillonnage...")
Na = 20
Te = 4000 * 0.00001
Ne = 4000
fs = Ne / Te
Te_fft = 1
Ne_fft = 25_000
fs_fft = Ne_fft / Te_fft
# Affichage des parametres d'echantillonnage
print("\nParametres d'echantillonnage (avec unites) :")
print(f"Te = {Te} s")
print(f"Ne = {Ne} echantillons")
print(f"fs = {fs:.6g} Hz")
print(f"Na = {Na}")
print("\nCalcul des tensions en reponse a la vibration de la corde...")

temps = np.linspace(0, Te, Ne, endpoint=False)
fem_s = mod.fem_singlecoil(Na, N_s, xm, temps, A, L, T, mu, x0, S, dBdy)
fem_h = mod.fem_humbucker(Na, N_h, x1, x2, temps, A, L, T, mu, x0, S, dBdy)
```

```
                                        #!####################!#
                                        #!       Exports      !#
                                        #!####################!#


# Ensure export directory exists
os.makedirs("Python/Exports", exist_ok=True)
print("-" * 100)


if tracer_fft:
    print("-" * 100)
    print("\nCalcul des FFT...")
    print("\nParametres d'echantillonnage (avec unites) :")
    print(f"Te = {Te_fft} s")
    print(f"Ne = {Ne_fft} echantillons")
    print(f"fs = {fs_fft:.6g} Hz")
    print(f"Na = {Na}")
    print("\nCalcul des tensions en reponse a la vibration de la corde...")

    temps_fft = Test_temps_h_fft
    fem_s_fft = mod.fem_singlecoil(Na, N_s, xm, temps_fft, A, L, T, mu, x0, S, dBdy)
    fem_h_fft = mod.fem_humbucker(Na, N_h, x1, x2, temps_fft, A, L, T, mu, x0, S, dBdy)

    fft_s_fem = mod.compute_and_normalize_fft(fem_s_fft, Ne_fft)
    fft_h_fem = mod.compute_and_normalize_fft(fem_h_fft, Ne_fft)
    fft_h_test = mod.compute_and_normalize_fft(Test_h_fft, len(Test_h_fft))
    fft_s_test = mod.compute_and_normalize_fft(Test_s_fft, len(Test_s_fft))


    freqs = np.fft.fftfreq(len(Test_h_fft), 1 / (len(Test_h_fft) / (Test_temps_h_fft[-1] - Test_temps_h_fft[0])))
    mask = 0 <= freqs
    freqs = freqs[mask]
    fft_s_fem = fft_s_fem[mask]
    fft_h_fem = fft_h_fem[mask]
    fft_h_test = fft_h_test[mask]
    fft_s_test = fft_s_test[mask]

    mask2 = freqs <= 800
    freqs = freqs[mask2]
    fft_s_fem = fft_s_fem[mask2]
    fft_h_fem = fft_h_fem[mask2]
    fft_h_test = fft_h_test[mask2]
    fft_s_test = fft_s_test[mask2]


    fft_s_fem = fft_s_fem / np.max(fft_s_fem)
    fft_h_fem = fft_h_fem / np.max(fft_h_fem)
    fft_h_test = fft_h_test / np.max(fft_h_test)
    fft_s_test = fft_s_test / np.max(fft_s_test)

    print("FFT calculees.")
    Title = "Spectre de la tension en sortie des micros\n(Fondamental à " + str(F) + " Hz)"
    print("Affichage des FFT...")

    #-- 01 --#
    mod.save_plot(PATH_exp + "FFT/Theorie.png",
            (freqs, fft_h_fem, {'color': Color_h, 'label': "Micro Humbucker - Theorique", 'linestyle': ls_th}),
            (freqs, fft_s_fem, {'color': Color_s, 'label': "Micro Singlecoil - Theorique", 'linestyle': ls_th}),
            title=Title, xlabel="Frequence [$Hz$]", ylabel="Tension normalisee")
    print(f"FFT sauvegardees sous {PATH_exp + "FFT/Theorie.png"}")

```

IV. Reponse_magnetique.py

```python
                                        #!####################!#
                                        #!   Bibliothèques    !#
                                        #!####################!#

print("\nImportation des bibliotheques...")
import os
import sys
sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__), '../..')))
from library import Modelisation as mod
import numpy as np
import matplotlib.pyplot as plt
import logging
plt.close("all")
logging.basicConfig(level=logging.INFO, format='%(levelname)s: %(message)s')
print("Bibliotheques importees.")

#! ------------------ Constantes --------------------------------
print("-" * 100)
print("\nInitialisation des constantes")
#* Constantes code
PATH_out = "Python/Exports/03-Reponse_magnetique/"
#* Constante physiqu
rho = 1.68 * 10**(-8)   # Masse volumique du cuivre (Ohm.m)
#* Parametre Micros
S = 7 * 10**(-4)        # Surface de la bobine du micro
d = 0.063 * 10**(-3)    # Diametre du fil de cuivre
xm = 0.105              # Position du micro singlecoil
x1 = 0.140              # Position du micro humbucker 1
x2 = 0.160              # Position du micro humbucker 2
L = 0.845               # Longueur de la corde
mu = 9 * 10**(-3)       # Masse lineique de la corde
dBdy = 0.03             #? Gradient du champ magnetique (A verifier ???)
#* Parametre Micro Singlecoil
RCC_s = 5.3 * 10**3     # Resistance DC micro singlecoil
lmoy_s = 0.147          #? Longueur moyenne du fil enroule
N_s = int(RCC_s * np.pi * d**2 / (4 * rho * lmoy_s))    # Nombre de spires du micro singlecoil
#* Parametre Micro Humbucker
RCC_h = 4.52 * 10**3    # Resistance DC micro humbucker
lmoy_h = lmoy_s         #? Longueur moyenne d'une bobine humbucker
N_h = 2*int(RCC_h * np.pi * d**2 / (8 * rho * lmoy_h))
#* Parametre Champ parasite
fB = 50                 # Frequence du champ parasite
B0 = 46 * 10**(-6)      # Champ magnetique au centre des bobines Helmholtz
alpha = 2               # Coefficient alpha
#* Couleurs graphiques
Color_h_test = "#008B8B"
Color_s_test = "#FF8C00"
m_exp="o"
ms_exp=5
Color_h = "#000080"
Color_s = "#8B0000"
```

```python
# ======================== Resultats / Execution ========================
print("Lecture des donnees de test...")
List_Bpar = np.array([0, 5.5, 11, 16.5, 22, 27.5, 33, 38.5, 44, 49.5]) * 10**-6
ks = N_s*S*2*np.pi*fB
kh = N_h*S*2*np.pi*fB*alpha*np.abs(x1-x2)
List_amp_h_th = np.array([kh * B for B in List_Bpar])
List_amp_s_th = np.array([ks * B for B in List_Bpar])

List_amp_h_test = [0]
List_amp_s_test = [0]

file = open("Python/data/filter/!Reponse_magnetique_amplitude.csv")
L_lignes = file.readlines()
for ligne in L_lignes[1:]:
    ligne = ligne.strip().split('\t')
    ligne = [x.replace('\n', '') for x in ligne]
    ligne = [x.replace(',', '.') for x in ligne]
    label = ligne[0]
    amp = float(ligne[1])
    if "Humbucker" in label:
        List_amp_h_test.append(amp - 0.005)
    elif "Single Coil" in label:
        List_amp_s_test.append(amp)

List_amp_h_test = np.array(List_amp_h_test)
List_amp_s_test = np.array(List_amp_s_test)

k_h_exp, _ = np.polyfit(List_Bpar, List_amp_h_test, 1)
k_s_exp, _ = np.polyfit(List_Bpar, List_amp_s_test, 1)
List_amp_h_test_lin = k_h_exp * List_Bpar
List_amp_s_test_lin = k_s_exp * List_Bpar
```

```python
doc_path = "Python/data/filter/SingleCoil_par_17,5V.csv"
try:
    with open(doc_path) as doc:
        L_lignes = doc.readlines()
    Test_temps_s_p = []
    Test_s_p = []
    for i in range(10001, 35001):
        values = L_lignes[i].strip().split("\t")
        values = [float(v.replace("\n", "")) for v in values]
        Test_temps_s_p.append(values[0])
        Test_s_p.append(values[1])
    Test_temps_s_p = np.array(Test_temps_s_p)
    Test_temps_s_p -= Test_temps_s_p[0]
    Test_s_p = np.array(Test_s_p)
    print("Donnees de test Single Coil parasite seules lues.")
except Exception as e:
    logging.error(f"Erreur lors de la lecture des donnees de test Single Coil parasite seules : {e}")

```

```python
print("\nCalcul des tensions induites au champ parasite...")
Te = 4000 * 0.00001
Ne = 4000
temps = np.linspace(0, Te, Ne, endpoint=False)
fem_s_par = mod.fem_par_s(N_s, fB, temps, B0, S, 2*np.pi/3)
fem_h_par = mod.fem_par_h(N_h, x1, x2, fB, temps, B0, alpha, S)


#* --- Micro SingleCoil (Parasite) ---*#
mod.save_plot(PATH_out + "Singlecoil.png",
        (temps, Test_s_p[:4000], {'marker': m_exp, 'markersize': ms_exp, 'linestyle': 'None', 'color': Color_s_test, 'label': "Micro Singlecoil - Experimental [$V$]"}),
        (temps, fem_s_par, {'color': Color_s, 'label': "Micro Singlecoil - Theorique [$V$]"}),
        title="Tension induite dans le micro SingleCoil \nsoumis à un champs magnetique de 50µT",
        xlabel="Temps [$s$]", ylabel="Tension [$V$]", ylim=(-0.5, 0.5))
print("Courbes sauvegardees sous Python/Exports/03-Parasite_singlecoil.png.")
```

```
CMRR_exp = 20*np.log10(k_s_exp / k_h_exp)
CMRR_th = 20*np.log10(ks/kh)
taux_th = ks/kh
taux_exp = k_s_exp / k_h_exp

eps_h = abs((k_h_exp - kh) / kh) * 100
eps_s = abs((k_s_exp - ks) / ks) * 100
eps = abs((taux_th - taux_exp) / (taux_th)) * 100
eps_cmrr = abs((CMRR_th - CMRR_exp)/(CMRR_th)) *100

print("\nCalcul des sensibilites et CMRR theorique")
print(f"k_s = {ks*10**-3} mV/microTesla")
print(f"k_h = {kh*10**-3} mV/microTesla")
print(f"Taux theorique = {taux_th}")
print(f"CMRR theorique = {20*np.log10(ks / kh)} dB")

print("\nCalcul des sensibilites et CMRR experimental")
print(f"k_s_exp = {k_s_exp*10**-3} mV/microTesla")
print(f"k_h_exp = {k_h_exp*10**-3} mV/microTesla")
print(f"Taux exp = {taux_exp}")
print(f"CMRR_exp = {20*np.log10( k_s_exp / k_h_exp )} dB")
print("\nCalcul des erreurs relatives...")

print(f"\nErreur relative k_h : {eps_h} %")
print(f"Erreur relative k_s : {eps_s} %")
print(f"Erreur relative taux : {eps} %")
print(f"Erreur relative cmrr : {eps_cmrr} %")
```